

Section 14.8: Industry Architecture Analysis

Part 8 of 8 in the System Design Interview Series

Executive Overview

Studying real production systems reveals how theoretical patterns translate under actual constraints. This section analyses architectures from Figma, Discord, Netflix, and compares message queue systems. The goal is not memorisation — it is pattern recognition: seeing *why* each company made the choices they did, and applying the same reasoning to novel interview questions.

1. Figma: Real-Time Collaboration Engine

The Problem Figma Solved

Google Docs used Operational Transform (OT) — a central server that transforms every operation. This works but creates a single point of failure and high latency for globally distributed users. Figma needed multi-user, multi-region, real-time editing for a design tool where objects are independent (not a linear text document).

CRDT-Based Architecture

Why CRDTs fit design tools:

In a text document, operations have positional dependencies (inserting at position 5 affects all positions > 5). In a design canvas, objects are independent (moving a rectangle doesn't affect a circle). This independence maps naturally to CRDTs.

```
CRDT design operations (all commutative):  
  User A: Move rectangle-42 to (100, 200)  
  User B: Change rectangle-42 fill to #FF0000
```

```
Both operations reference the object by immutable UUID, not position.  
Apply in any order → same result:  
  rectangle-42 at (100, 200) with fill #FF0000 ✓
```

Last-write-wins for conflicts: If both users set position simultaneously, the higher-timestamp write wins. For design tools, this is acceptable — the conflict resolution is visible and users can undo.

Operational Pipeline

```
User action (drag object) →  
  Local CRDT operation applied immediately (optimistic) →  
  Operation queued to server →  
  Server broadcasts to all connected clients →  
  Clients apply to their CRDT state →  
  All clients converge to identical state
```

Persistence:
 Snapshot every 30 seconds (full document state)
 + Operation log between snapshots
 Recovery: load snapshot + replay log

FlatBuffers for efficiency: Figma uses FlatBuffers (not JSON) for operation serialisation. FlatBuffers allows reading fields without full deserialisation. A 100-object canvas update serialises in ~50µs vs ~5ms for JSON. At 60 ops/second per user × thousands of concurrent users, this matters significantly.

Scale Insights and Lessons

Challenge	Solution	Lesson
Global latency	50+ edge locations via Cloudflare	Co-locate operation broadcast with users
Offline editing	Local operation log replayed on reconnect	CRDTs make offline trivial — just replay
Large documents	Incremental snapshot diff	Don't snapshot the full state every time
Tombstone accumulation	Background GC job	CRDTs generate garbage; plan for cleanup

Key lesson: CRDTs are not magic. Operations must be designed carefully to be commutative. An operation like “set z-index to 5” is not commutative if two users set z-index simultaneously. Figma handles this with tie-breaking (timestamp + user ID).

2. Discord: Trillions of Messages — Cassandra to ScyllaDB Migration

The Problem

Discord reached the limits of their Cassandra cluster: 600+ nodes, 100TB+ of message data, and degrading read performance under hot-shard conditions. They migrated to ScyllaDB — a C++ reimplementation of Cassandra's API with better performance characteristics.

Why This Migration Is a Great Interview Case Study

It demonstrates: - How to migrate a live system with zero downtime - The dual-write pattern (applicable to any DB migration) - What “hot sharding” looks like in practice and how to mitigate it

Dual-Write Migration Pattern

Phase 1 — Dual Write:

- Application writes to BOTH Cassandra AND ScyllaDB
- Reads still served from Cassandra only
- ScyllaDB verified via background consistency checker

Phase 2 — Read Shadowing:

- Application sends all reads to BOTH databases
- Returns Cassandra result to user
- Compares ScyllaDB result in background, logs discrepancies

Phase 3 — Read Cutover (per shard):

- Route 1% of reads to ScyllaDB → monitor error rate
- Ramp to 10%, 50%, 100% over weeks
- Rollback path: one config change reverts to Cassandra

Phase 4 — Write Cutover:

- Stop writing to Cassandra
- Cassandra becomes read-only backup

Phase 5 — Decommission:

- Delete Cassandra cluster after retention period

Why shard-by-shard? A full cutover risks a catastrophic rollback. Shard-by-shard limits blast radius. If ScyllaDB has a problem with one shard's access pattern, only that shard reverts, not the entire system.

Hot Sharding Problem and Solution

Discord's message schema partitioned by `channel_id`. A popular channel (millions of members) generates millions of messages per day — all on one partition. This creates a “hot” partition that overwhelms a single node.

Symptoms of hot sharding:

- P99 read latency spikes on a few nodes
- CPU and I/O maxed on hot partition nodes
- Other partitions on the same node degrade

Mitigations:

1. Sub-partition: channel_id + bucket_id (time bucket per day)
→ Distributes one channel's data across multiple partitions
2. Rate limiting: Limit writes to a channel to prevent runaway growth
3. Manual overrides: Manually reassign celebrity channels to dedicated nodes

Lessons Applied to Interview Questions

1. **Never “flip the switch”**: All migrations need dual-write + gradual cutover phases
2. **Observability first**: Before migrating, instrument everything. You need dashboards for read latency, error rate, and data consistency metrics per shard
3. **Access patterns evolve**: The schema you designed for 10M users often breaks at 500M users. Design for your next 10x growth, not just current scale
4. **Hotspot mitigation is a first-class concern**: Any partition key that correlates with user activity (user_id, channel_id, video_id) can become a hotspot

3. Netflix: Microservices & Chaos Engineering

Multi-Region Active-Active Architecture

Netflix operates every region as an independent unit. There is no primary-secondary failover — every region serves live traffic, and the system is designed to survive complete region loss.

US-East, US-West, EU-West (and more) all simultaneously:

- Serve user requests from their region
- Maintain their own copy of user state (watch history, preferences)
- Sync asynchronously via Kafka replication

Region failure scenario:

US-East loses connectivity →
DNS health checks detect failure within 60 seconds →
Traffic routed to US-West and EU-West →
Users experience brief interruption; then normal service
No manual intervention required

Why active-active over active-passive? In active-passive, the passive region is untested under load until it becomes primary — and that's the worst time to discover it doesn't work. Active-active regions are always serving real traffic, so their capacity and correctness are continuously validated.

Chaos Engineering

Netflix's most influential engineering contribution is chaos engineering — deliberately injecting failures in production to validate that systems degrade gracefully.

```
Chaos Monkey: Randomly terminates EC2 instances in production
→ Engineers must design services to survive instance death
→ On-call engineers learn to respond to realistic incidents
```

```
Chaos Kong: Terminates an entire availability zone
→ Services must handle AZ failure without data loss
```

```
Latency injection: Adds artificial delay to service calls
→ Reveals tight coupling and missing timeouts
```

```
Result: Engineers treat failure as normal, not exceptional
```

Interview application: When asked “what happens if service X fails?”, Netflix-style answers describe automatic degradation paths, not manual recovery. “The recommendation service falls back to pre-computed popular titles. Users see slightly stale recommendations, not an error page.”

Recommendation Pipeline Architecture

Netflix's two-stage recommendation system (covered technically in 14.5) with the production infrastructure detail:

```
Offline (daily):
  Hadoop clusters process viewing history → Matrix factorisation →
  User embeddings + Item embeddings →
  Pre-computed candidate lists per user → EVCache (Redis-based)

Online (per request, < 100ms):
  Fetch pre-computed candidates from EVCache →
  Run lightweight ranking model (features: time of day, device, trending) →
  Return personalised row ordering
```

EVCache: Netflix's Redis-based distributed cache, with a key design choice — data is replicated across AZs within a region. A cache miss is served from another AZ's copy, not from the origin. This makes cache a near-durable store, not a soft dependency.

4. Kafka vs. RabbitMQ vs. ActiveMQ — When to Use Each

This comparison is one of the most common infrastructure interview questions. The answer should always start with: “It depends on your requirements.”

Architectural Philosophy Comparison

Dimension	Kafka	RabbitMQ	ActiveMQ
Model	Log-based (consumers pull from append-only log)	Broker-based (broker pushes to consumers)	JMS broker (push)
Message retention	Configurable (days/weeks)	Until consumed + ACK	Until consumed
Throughput	Highest (100K–1M msg/s)	Medium (10–50K msg/s)	Lower (1–10K msg/s)
Latency	Medium (10–100ms)	Lowest (1–10ms)	Medium
Ordering	Per partition (key-based)	Per queue (strict)	Per queue
Consumer groups	Multiple independent groups at no extra cost	Fan-out via exchange bindings	Fan-out via topics
Replay	Yes — rewind offset to any point	No — messages gone after ACK	No
Ecosystem	Kafka Streams, ksqldb, Flink integration	Plugin ecosystem	Enterprise connectors

Decision Framework: When to Use Which

Use Kafka when: - You need replay capability (audit, debugging, re-processing) - Multiple downstream systems consume the same events independently - You need high throughput (> 50K msg/s) - You need stream processing (Kafka Streams, Flink) - Event sourcing or CQRS architecture

Use RabbitMQ when: - You need the lowest latency (< 5ms) - Complex routing logic (exchange types: direct, topic, fanout, headers) - RPC-style request-reply patterns - Work queues with competing consumers and acknowledgement - Your throughput is moderate (< 50K msg/s)

Use ActiveMQ when: - Integrating with legacy Java enterprise systems (JMS) - You need guaranteed message ordering with transactions - Enterprise integration patterns (EIP) out of the box

Kafka for Payment Systems — The Trade-off Analysis

Why Kafka is often preferred for payments:

Payment event flow with Kafka:
Payment API → Kafka topic "payments"

Multiple consumers, all independent:

Consumer A (fraud detection):	reads "payments", writes fraud scores
Consumer B (accounting):	reads "payments", updates GL
Consumer C (notifications):	reads "payments", sends receipts
Consumer D (analytics):	reads "payments", updates dashboards

With RabbitMQ:

Payment event consumed by one consumer queue
Must fan-out via exchange bindings
Message deleted after consumption → no replay
If accounting was down for 2 hours, those events are lost

The replay argument (key for payments):

Scenario: A bug in the accounting service caused incorrect GL entries for 6 hours yesterday.

With Kafka (retention = 7 days):

Fix the bug → restart accounting consumer → rewind offset to 6 hours ago
→ Replay all payments → GL corrected automatically
Time to fix: 2 hours

Without Kafka (RabbitMQ, no retention):

No replay possible
Must reconstruct from payment DB logs manually
Time to fix: 2 days + manual reconciliation risk

The latency trade-off: Kafka adds 10–100ms vs RabbitMQ's 1–10ms. For a payment that already takes 500ms+ due to bank network latency, this is irrelevant. Kafka's durability and replay capability outweigh the latency difference.

Interview Problem 1 — Message Queue Selection

“Your team is building an order processing system for an e-commerce platform. Orders need to trigger: inventory reservation, payment processing, warehouse notification, and customer email. Which message queue do you use and why?”

Solution framework:

1. **Choose Kafka** — here's the reasoning:

- Four independent consumers need the same order event (fan-out naturally supported by consumer groups)

- If warehouse notification fails and their consumer crashes, they can replay missed orders from Kafka after recovery — no orders lost
- Order events are business-critical records; 7-day retention means a full audit trail
- Future analytics (what's our order volume by hour?) can consume the same topic

2. **Kafka topic design:**

```
Topic: order-events (partitioned by order_id for ordering)
Consumer Group 1: inventory-service
Consumer Group 2: payment-service
Consumer Group 3: warehouse-service
Consumer Group 4: notification-service
```

3. **RabbitMQ would work but with caveats:** You'd need a fanout exchange, 4 queues, and each service would need its own dead-letter queue. If a service is down, messages accumulate in its queue or are lost depending on TTL settings. No replay after the queue is empty.
4. **Trade-off stated:** If lowest latency (< 5ms) were critical (e.g., high-frequency trading signals), you'd choose RabbitMQ. For an order processing system where 50ms is fine and durability matters, Kafka wins.

Interview Problem 2 — Database Migration with Zero Downtime

“Your startup's Postgres database is hitting its limits. You need to migrate 500GB of user data to a new sharded Cassandra cluster without any downtime. Walk through your plan.”

Solution framework (applying Discord's dual-write lessons):

1. **Phase 0 — Observability:** Before touching anything, instrument every DB query with latency metrics and set up dashboards. You need a baseline to compare against post-migration.
2. **Phase 1 — Dual write** (week 1):
 - Add a write adapter that writes every new record to both Postgres and Cassandra simultaneously
 - Reads still served from Postgres
 - Cassandra is write-only, not yet trusted
3. **Phase 2 — Backfill** (week 1–2):
 - Run a background migration job that reads historical records from Postgres and writes to Cassandra in batches
 - Rate-limit to avoid overwhelming either DB (e.g., 10K records/second)

- Track progress; the job is idempotent (re-runnable)

4. **Phase 3 — Consistency verification** (week 2):

- For every read from Postgres, shadow-read the same key from Cassandra
- Log discrepancies (don't act on them — just measure)
- Target: discrepancy rate < 0.01% before proceeding

5. **Phase 4 — Read cutover** (week 3):

- Route 1% of reads to Cassandra → check error rate and latency
- Gradually ramp: 5% → 20% → 50% → 100% over 1 week
- Instant rollback: one config flag routes traffic back to Postgres

6. **Phase 5 — Write cutover** (week 4):

- Stop writing to Postgres
- Postgres becomes read-only backup for 30 days (rollback window)

7. **Phase 6 — Decommission** (week 8):

- Delete Postgres after 30-day retention period
- Celebrate

Section Summary: Industry Architecture Lessons

Company	Key Pattern	Transferable Lesson
Figma	CRDT for collaborative editing	Design operations to be commutative; plan for tombstone GC
Discord	Dual-write migration, hot shard mitigation	Never flip the switch; sub-partition hot keys
Netflix	Active-active multi-region + chaos engineering	Test failure in production; design for graceful degradation
Kafka vs RabbitMQ	Log-based vs broker-based	Choose by replay/fan-out needs (Kafka) vs latency/routing (RabbitMQ)

Complete Series Reference: When to Apply Each Pattern

Pattern	Apply When	Avoid When
Fan-out-on-write	Small follower counts (< 10K)	Celebrity accounts
Fan-out-on-read	Celebrity accounts, large groups	High read frequency (timeline)
Saga	Distributed transactions across services	Simple single-DB transactions
2PC	Must-be-atomic, small participant count	High availability required
CRDT	Offline-first, no central authority	Non-commutative operations
Consistent hashing	Dynamic node add/remove	Small, fixed cluster
Fencing tokens	Distributed lock with data store writes	Lock is advisory only
Dual-write migration	Live DB migration, zero downtime	Short maintenance windows acceptable
Chaos engineering	Validating resilience at scale	Pre-production systems
Geohash	Distributed proximity queries	Complex polygon shapes (use S2)